Lab Assignment 13

1. (Refactoring – Removing Code Duplication)

   Task:

   - Use AI to refactor a given Python script that contains multiple repeated code blocks.

   Instructions:

   - Prompt AI to identify duplicate logic and replace it with functions or classes.
   - Ensure the refactored code maintains the same output.
   - Add docstrings to all functions.

   Sample Legacy Code:

   # Legacy script with repeated logic
   print("Area of Rectangle:", 5 * 10)
   print("Perimeter of Rectangle:", 2 * (5 + 10))
   print("Area of Rectangle:", 7 * 12)
   print("Perimeter of Rectangle:", 2 * (7 + 12))
   print("Area of Rectangle:", 10 * 15)
   print("Perimeter of Rectangle:", 2 * (10 + 15))

   Expected Output:

   - Refactored code with a reusable function and no duplication.
   - Well documented code

1.py ×

Sem-VI > AI-Assisted > Ass_13 > 1.py > rectangle_area

```python
def rectangle_area(length: float, width: float) -> float:
    """
    Calculate the area of a rectangle.

    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.

    Returns:
        float: The area of the rectangle.
    """
    return length * width

def rectangle_perimeter(length: float, width: float) -> float:
    """
    Calculate the perimeter of a rectangle.

    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.

    Returns:
        float: The perimeter of the rectangle.
    """
    return 2 * (length + width)

print("Area of Rectangle:", rectangle_area(5, 10))
print("Perimeter of Rectangle:", rectangle_perimeter(5, 10))
print("Area of Rectangle:", rectangle_area(7, 12))
print("Perimeter of Rectangle:", rectangle_perimeter(7, 12))
print("Area of Rectangle:", rectangle_area(10, 15))
print("Perimeter of Rectangle:", rectangle_perimeter(10, 15))
```

2. (Refactoring – Extracting Reusable Functions)

Task:

- Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.

Instructions:

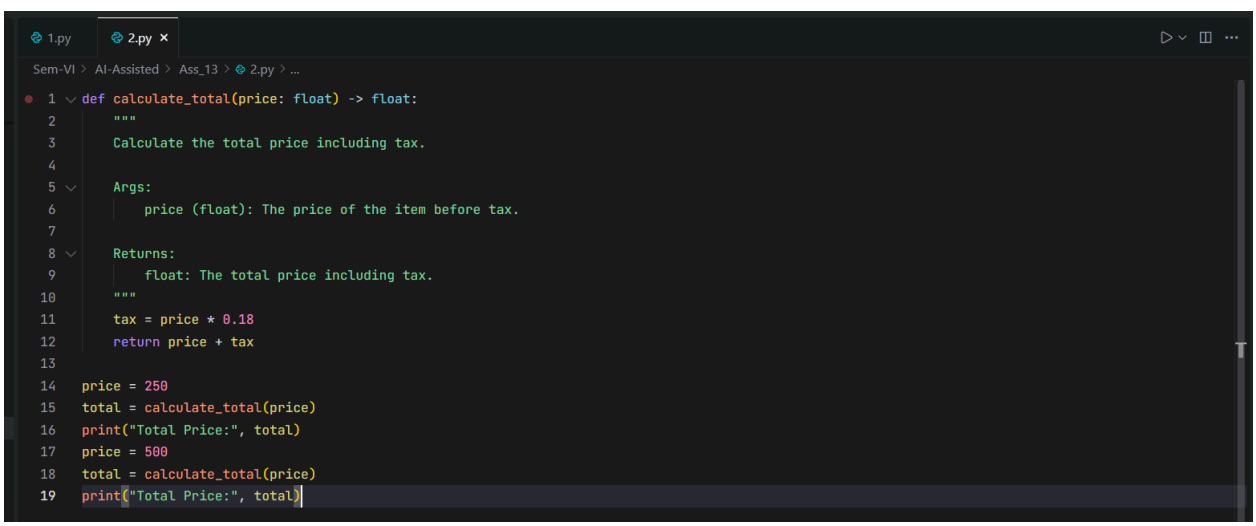
- Identify repeated or related logic and extract it into reusable functions.
- Ensure the refactored code is modular, easy to read, and documented with docstrings.

Sample Legacy Code:

```
# Legacy script with inline repeated logic
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)
price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

Expected Output:

- Code with a function calculate_total(price) that can be reused for multiple price inputs.
- Well documented code



3. Refactoring Using Classes and Methods (Eliminating Redundant Conditional Logic)

- Refactor a Python script that contains repeated if–elif–else grading logic by implementing a structured, object-oriented solution using a class and a method.

Problem Statement

- The given script contains duplicated conditional statements used to assign grades based on student marks. This redundancy violates clean code principles and reduces maintainability.
- You are required to refactor the script using a class-based design to improve modularity, reusability, and readability while preserving the original grading logic.

Mandatory Implementation Requirements

1. Class Name: GradeCalculator
2. Method Name: calculate_grade(self, marks)
3. The method must:
    a. Accept marks as a parameter.
    b. Return the corresponding grade as a string.
    c. The grading logic must strictly follow the conditions below:
        i. Marks ≥ 90 and ≤ 100 → "Grade A"
        ii. Marks ≥ 80 → "Grade B"
        iii. Marks ≥ 70 → "Grade C"
        iv. Marks ≥ 40 → "Grade D"
        v. Marks ≥ 0 → "Fail"

    Note: Assume marks are within the valid range of 0 to 100.
4. Include proper docstrings for:
    a. The class
    b. The method (with parameter and return descriptions)
5. The method must be reusable and called multiple times without rewriting conditional logic.

Given code:

```
marks = 85
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
marks = 72
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")
```

Expected Output:

- Define a class named GradeCalculator.
- Implement a method calculate_grade(self, marks) inside the class.
- Create an object of the class.
- Call the method for different student marks.

- Print the returned grade values.

```python
class GradeCalculator:
    def calculate_grade(self, marks: int) -> str:
        """
        Calculate the grade based on the marks.

        Args:
            marks (int): The marks of the student.

        Returns:
            str: The grade of the student.
        """
        if marks >= 90:
            return "Grade A"
        elif marks >= 75:
            return "Grade B"
        else:
            return "Grade C"

grade_calculator = GradeCalculator()
marks = 85
print(grade_calculator.calculate_grade(marks))
marks = 72
print(grade_calculator.calculate_grade(marks))
```

4. (Refactoring – Converting Procedural Code to Functions)

   Task: Use AI to refactor procedural input–processing logic into functions.

   Instructions:
   - Identify input, processing, and output sections.
   - Convert each into a separate function.
   - Improve code readability without changing behavior.

   Sample Legacy Code:

```python
num = int(input("Enter number: "))
square = num * num
print("Square:", square)
```

   Expected Output:
   - Modular code using functions like get_input(), calculate_square(), and display_result().

1.py 2.py 3.py 4.py ×

Sem-VI > AI-Assisted > Ass_13 > 4.py > get_input

```python
def get_input() -> int:
    """
    Get the number from the user.

    Returns:
        int: The number entered by the user.
    """
    return int(input("Enter number: "))

def calculate_square(num: int) -> int:
    """
    Calculate the square of the number.

    Args:
        num (int): The number to calculate the square of.

    Returns:
        int: The square of the number.
    """
    return num * num

def display_result(square: int) -> None:
    """
    Display the square of the number.

    Args:
        square (int): The square of the number.
    """
    print("Square:", square)

num = get_input()
square = calculate_square(num)
display_result(square)
```

5. (Refactoring Procedural Code into OOP Design)

   Task: Use AI to refactor procedural code into a class-based design.

   Focus Areas:

   - Object-Oriented principles
   - Encapsulation

   Legacy Code:

```python
salary = 50000
tax = salary * 0.2
net = salary - tax
print(net)
```

   Expected Outcome:

   - A class like EmployeeSalaryCalculator with methods and attributes.

5.py

Sem-VI > AI-Assisted > Ass_13 > 5.py > EmployeeSalaryCalculator

```python
class EmployeeSalaryCalculator:
    def __init__(self, salary: float):
        self.salary = salary

    def calculate_net_salary(self) -> float:
        tax = self.salary * 0.2
        return self.salary - tax

    def display_net_salary(self) -> None:
        net = self.calculate_net_salary()
        print("Net Salary:", net)

emp1sal = EmployeeSalaryCalculator(50000)
emp1sal.display_net_salary()
```

6. (Optimizing Search Logic)

   Task: Refactor inefficient linear searches using appropriate data structures.

   Focus Areas:

   - Time complexity
   - Data structure choice

   Legacy Code:

```python
users = ["admin", "guest", "editor", "viewer"]
name = input("Enter username: ")
found = False
for u in users:
if u == name:
found = True
print("Access Granted" if found else "Access Denied")
```

   Expected Outcome:

   - Use of sets or dictionaries with complexity justification

6.py

Sem-VI > AI-Assisted > Ass_13 > 6.py > users

```python
users = {"admin", "guest", "editor", "viewer"}
name = input("Enter username: ")

if name in users:
    print("Access Granted")
else:
    print("Access Denied")
```

Justification:

Rather than using a general linear search with a list we can convert the list into a dictionary set, where when used lookup the time complexity is O(1) rather than the complexity of O(n).

7. Refactoring the Library Management System

   You are provided with a poorly structured Library Management script that:

   - Contains repeated conditional logic

- Does not use reusable functions
- Lacks documentation
- Uses print-based procedural execution
- Does not follow modular programming principles

Your task is to refactor the code into a proper format

1. Create a module library.py with functions:
    a. add_book(title, author, isbn)
    b. remove_book(isbn)
    c. search_book(isbn)
2. Insert triple quotes under each function and let Copilot complete the docstrings.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format.
5. Open the file in a browser.
6. Given Code

```
# Library Management System (Unstructured Version)
# This code needs refactoring into a proper module with documentation.
library_db = {}

# Adding first book
title = "Python Basics"
author = "John Doe"
isbn = "101"
if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")

# Adding second book (duplicate logic)
title = "AI Fundamentals"
author = "Jane Smith"
isbn = "102"
if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")

# Searching book (repeated logic structure)
isbn = "101"
```

```python
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")

# Removing book (again repeated pattern)
isbn = "101"
if isbn in library_db:
    del library_db[isbn]
    print("Book removed successfully.")
else:
    print("Book not found.")

# Searching again
isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")
```

6.py library.py ×

Sem-VI > AI-Assisted > Ass_13 > library.py > search_book

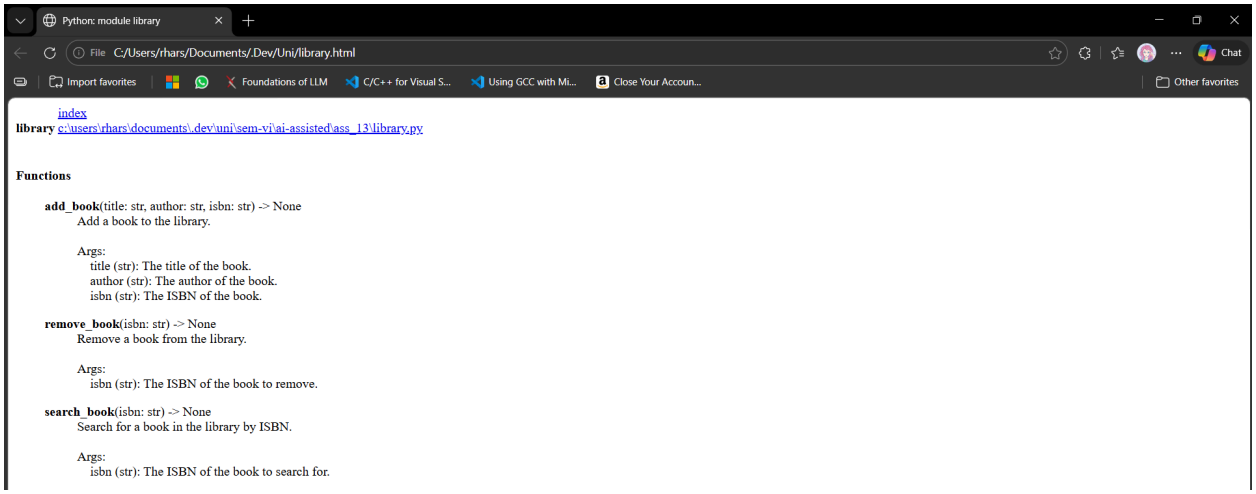
```python
def add_book(title: str, author: str, isbn: str) -> None:
    """
    Add a book to the library.

    Args:
        title (str): The title of the book.
        author (str): The author of the book.
        isbn (str): The ISBN of the book.
    """
    # Add the book to the library
    print(f"Book '{title}' by {author} with ISBN {isbn} added to the library.")

def remove_book(isbn: str) -> None:
    """
    Remove a book from the library.

    Args:
        isbn (str): The ISBN of the book to remove.
    """
    # Remove the book from the library
    print(f"Book with ISBN {isbn} removed from the library.")

def search_book(isbn: str) -> None:
    """
    Search for a book in the library by ISBN.

    Args:
        isbn (str): The ISBN of the book to search for.
    """
    # Search for the book in the library
    print(f"Searching for book with ISBN {isbn}...")
    # If the book is found, display its details
    print(f"Book with ISBN {isbn} found: Title, Author, Year Published")
    # If the book is not found, display a message
    print(f"Book with ISBN {isbn} not found in the library.")
```

Generating HTML:

Problems Output Debug Console Terminal powershell

```
PS C:\Users\rhars\Documents\.Dev\Uni> python -m pydoc -w C:\Users\rhars\Documents\.Dev\Uni\Sem-VI\AI-Assisted\Ass_13\library.py
wrote library.html
PS C:\Users\rhars\Documents\.Dev\Uni>
```

HTML Documentation:



index
library c:\users\rhars\documents\.dev\uni\sem-vi\ai-assisted\ass_13\library.py

Functions

add_book(title: str, author: str, isbn: str) -> None
Add a book to the library.

Args:
title (str): The title of the book.
author (str): The author of the book.
isbn (str): The ISBN of the book.

remove_book(isbn: str) -> None
Remove a book from the library.

Args:
isbn (str): The ISBN of the book to remove.

search_book(isbn: str) -> None
Search for a book in the library by ISBN.

Args:
isbn (str): The ISBN of the book to search for.

8. Fibonacci Generator:

Write a program to generate Fibonacci series up to n.

The initial code has:

- Global variables.
- Inefficient loop.
- No functions or modularity.

Task for Students:

- Refactor into a clean reusable function (generate_fibonacci).
- Add docstrings and test cases.
- Compare AI-refactored vs original.

Bad Code Version:

```
# fibonacci bad version
n = int(input("Enter limit: "))
a = 0
b = 1
print(a)
print(b)

for i in range(2, n):
    c = a + b
    print(c)
    a = b
    b = c
```

8.py

Sem-VI > AI-Assisted > Ass_13 > 8.py > generate_fibonacci

```python
def generate_fibonacci(n: int) -> list:
    """
    Generate Fibonacci sequence up to the nth term.

    Args:
        n (int): The number of terms in the Fibonacci sequence.
    """
    a, b = 0, 1
    sequence = [a, b]
    for _ in range(2, n):
        c = a + b
        sequence.append(c)
        a, b = b, c
    return sequence

# Test cases:
assert generate_fibonacci(5) == [0, 1, 1, 2, 3]
assert generate_fibonacci(10) == [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Refactored	Original
Defined in a function helps in easier debugging	Declared Global Variables

9. Twin Primes Checker

Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).

The initial code has:

- Inefficient prime checking.
- No functions.
- Hardcoded inputs.

Task for Students:

- Refactor into is_prime(n) and is_twin_prime(p1, p2).
- Add docstrings and optimize.
- Generate a list of twin primes in a given range using AI.

Bad Code Version:

```python
# twin primes bad version
a = 11
b = 13
fa = 0

for i in range(2, a):
    if a % i == 0:
        fa = 1

fb = 0
```

```python
for i in range(2, b):
    if b % i == 0:
        fb = 1

if fa == 0 and fb == 0 and abs(a - b) == 2:
    print("Twin Primes")
else:
    print("Not Twin Primes")
```

9.py

Sem-VI > AI-Assisted > Ass_13 > 9.py > is_prime

```python
def is_prime(n: int) -> bool:
    """
    Check if a number is prime.

    Args:
        n (int): The number to check.

    Returns:
        bool: True if the number is prime, False otherwise.
    """
    # Check if the number is less than 2
    if n < 2:
        return False
    # Check if the number is divisible by any number from 2 to the square root of the number
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True

def is_twin_prime(p1: int, p2: int) -> bool:
    """
    Check if two numbers are twin primes.

    Args:
        p1 (int): The first prime number.
        p2 (int): The second prime number.

    Returns:
        bool: True if the numbers are twin primes, False otherwise.
    """
    # Check if both numbers are prime
    if not is_prime(p1) or not is_prime(p2):
        return False
    # Check if the difference between the numbers is 2
    if abs(p1 - p2) != 2:
        return False
    return True

a = int(input("Enter first prime number: "))
b = int(input("Enter second prime number: "))
if is_twin_prime(a, b):
    print("Twin Primes")
else:
    print("Not Twin Primes")
```

10. Refactoring the Chinese Zodiac Program

Objective

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

The current program reads a year from the user and prints the corresponding Chinese Zodiac sign. However, the implementation contains repetitive conditional logic, lacks modular design, and does not follow clean coding principles.

Your task is to refactor the code to improve readability, maintainability, and structure.

Chinese Zodiac Cycle (Repeats Every 12 Years)

1. Rat
2. Ox

3. Tiger
4. Rabbit
5. Dragon
6. Snake
7. Horse
8. Goat (Sheep)
9. Monkey
10. Rooster
11. Dog
12. Pig

```python
# Chinese Zodiac Program (Unstructured Version)
# This code needs refactoring.

year = int(input("Enter a year: "))

if year % 12 == 0:
    print("Monkey")
elif year % 12 == 1:
    print("Rooster")
elif year % 12 == 2:
    print("Dog")
elif year % 12 == 3:
    print("Pig")
elif year % 12 == 4:
    print("Rat")
elif year % 12 == 5:
    print("Ox")
elif year % 12 == 6:
    print("Tiger")
elif year % 12 == 7:
    print("Rabbit")
elif year % 12 == 8:
    print("Dragon")
elif year % 12 == 9:
    print("Snake")
elif year % 12 == 10:
    print("Horse")
elif year % 12 == 11:
    print("Goat")
```

You must:

1. Create a reusable function: get_zodiac(year)

2. Replace the if-elif chain with a cleaner structure (e.g., list or dictionary).
3. Add proper docstrings.
4. Separate input handling from logic.
5. Improve readability and maintainability.
6. Ensure output remains correct.

10.py

Sem-VI > AI-Assisted > Ass_13 > 10.py > ...

```python
def get_zodiac(year: int) -> str:
    """
    Get the Chinese Zodiac sign for a given year.

    Args:
        year (int): The year to get the zodiac sign for.

    Returns:
        str: The Chinese Zodiac sign.
    """
    # Define the Chinese Zodiac cycle
    zodiac_cycle = [
        "Rat", "Ox", "Tiger", "Rabbit", "Dragon", "Snake",
        "Horse", "Goat", "Monkey", "Rooster", "Dog", "Pig"
    ]
    # Get the index of the zodiac sign in the cycle
    index = year % 12
    # Return the zodiac sign
    return zodiac_cycle[index]

def main():
    """
    Main function to handle user input and display the Chinese Zodiac sign.
    """
    try:
        year = int(input("Enter a year: "))
        zodiac_sign = get_zodiac(year)
        print(f"The Chinese Zodiac sign for the year {year} is: {zodiac_sign}")
    except ValueError:
        print("Invalid input. Please enter a valid year.")
if __name__ == "__main__":
    main()
```

11. Refactoring the Harshad (Niven) Number Checker

- Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.
- A Harshad (Niven) number is a number that is divisible by the sum of its digits.

For example:

- 18 → 1 + 8 = 9 → 18 ÷ 9 = 2 □ (Harshad Number)
- 19 → 1 + 9 = 10 → 19 ÷ 10 ≠ integer □ (Not Harshad)

Problem Statement

The current implementation:

- Mixes logic and input handling
- Uses redundant variables
- Does not use reusable functions properly
- Returns print statements instead of boolean values
- Lacks documentation

You must refactor the code to follow clean coding principles.

```python
# Harshad Number Checker (Unstructured Version)
num = int(input("Enter a number: "))
temp = num
sum_digits = 0

while temp > 0:
    digit = temp % 10
    sum_digits = sum_digits + digit
    temp = temp // 10

if sum_digits != 0:
    if num % sum_digits == 0:
        print("True")
    else:
        print("False")
else:
    print("False")
```

You must:

1. Create a reusable function: is_harshad(number)
2. The function must:
   - Accept an integer parameter.
   - Return True if the number is divisible by the sum of its digits.
   - Return False otherwise.
3. Separate user input from core logic.
4. Add proper docstrings.
5. Improve readability and maintainability.
6. Ensure the program handles edge cases (e.g., 0, negative numbers).

10.py 11.py

Sem-VI > AI-Assisted > Ass_13 > 11.py > sum_of_digits

```python
def sum_of_digits(number: int) -> int:
    """
    Calculate the sum of the digits of a given number.

    Args:
        number (int): The number to calculate the sum of digits for.

    Returns:
        int: The sum of the digits.
    """
    total = 0
    while number > 0:
        total += number % 10
        number //= 10
    return total

def is_harshad(number: int) -> bool:
    """
    Check if a number is a Harshad (Niven) number.

    A Harshad number is divisible by the sum of its digits.

    Args:
        number (int): The number to check.
    Returns:
        bool: True if the number is a Harshad number, False otherwise.
    """
    if number <= 0:
        return False
    digit_sum = sum_of_digits(number)
    return digit_sum != 0 and number % digit_sum == 0

def main():
    """
    Main function to handle user input and display whether the number is a Harshad number.
    """
    try:
        num = int(input("Enter a number: "))
        if is_harshad(num):
            print(f"{num} is a Harshad number.")
        else:
            print(f"{num} is not a Harshad number.")
    except ValueError:
        print("Invalid input. Please enter a valid integer.")
if __name__ == "__main__":
    main()
```

12. Refactoring the Factorial Trailing Zeros Program

- Refactor the given poorly structured Python script into a clean, modular, and efficient implementation.
- The program calculates the number of trailing zeros in n! (factorial of n).

Problem Statement

The current implementation:

- Calculates the full factorial (inefficient for large n)
- Mixes input handling with business logic
- Uses print statements instead of return values
- Lacks modular structure and documentation

You must refactor the code to improve efficiency, readability, and maintainability.

```python
# Factorial Trailing Zeros (Unstructured Version)
n = int(input("Enter a number: "))
fact = 1
i = 1

while i <= n:
    fact = fact * i
    i = i + 1
```

```python
count = 0
while fact % 10 == 0:
    count = count + 1
    fact = fact // 10

print("Trailing zeros:", count)
```

You must:

1. Create a reusable function: count_trailing_zeros(n)
2. The function must:
   - Accept a non-negative integer n.
   - Return the number of trailing zeros in n!.
3. Do NOT compute the full factorial.
4. Use an optimized mathematical approach (count multiples of 5).
5. Add proper docstrings.
6. Separate user interaction from core logic.
7. Handle edge cases (e.g., negative numbers, zero).

10.py 11.py 12.py

Sem-VI > AI-Assisted > Ass_13 > 12.py > count_trailing_zeros

```python
def count_trailing_zeros(n: int) -> int:
    """
    Calculate the number of trailing zeros in n! (factorial of n).

    Args:
        n (int): A non-negative integer for which to calculate the trailing zeros.

    Returns:
        int: The number of trailing zeros in n!.
    """
    if n < 0:
        raise ValueError("Input must be a non-negative integer.")

    count = 0
    power_of_5 = 5
    while power_of_5 <= n:
        count += n // power_of_5
        power_of_5 *= 5
    return count

def main():
    """
    Main function to handle user input and display the number of trailing zeros in n!.
    """
    try:
        n = int(input("Enter a non-negative integer: "))
        if n < 0:
            print("Please enter a non-negative integer.")
            return
        zeros = count_trailing_zeros(n)
        print(f"The number of trailing zeros in {n}! is: {zeros}")
    except ValueError:
        print("Invalid input. Please enter a valid non-negative integer.")
if __name__ == "__main__":
    main()
```

13. (Collatz Sequence Generator – Test Case Design)

- Function: Generate Collatz sequence until reaching 1.

- Test Cases to Design:
    - Normal: 6 → [6,3,10,5,16,8,4,2,1]
    - Edge: 1 → [1]
    - Negative: -5
    - Large: 27 (well-known long sequence)
        - Requirement: Validate correctness with pytest.

Explanation:

We need to write a function that:

- Takes an integer n as input.
- Generates the Collatz sequence (also called the 3n+1 sequence).
- The rules are:
    - If n is even → next = n / 2.
    - If n is odd → next = 3n + 1.
- Repeat until we reach 1.
- Return the full sequence as a list.

Example

Input: 6

Steps:

- 6 (even → 6/2 = 3)
- 3 (odd → 3*3+1 = 10)
- 10 (even → 10/2 = 5)
- 5 (odd → 3*5+1 = 16)
- 16 (even → 16/2 = 8)
- 8 (even → 8/2 = 4)
- 4 (even → 4/2 = 2)
- 2 (even → 2/2 = 1)

Output:

- [6, 3, 10, 5, 16, 8, 4, 2, 1]

10.py 11.py 12.py 13.py

Sem-VI > AI-Assisted > Ass_13 > 13.py > collatz_sequence

```python
def collatz_sequence(n: int) -> list:
    """
    Generate the Collatz sequence for a given integer n.

    Args:
        n (int): The starting integer for the Collatz sequence.
    Returns:
        list: The Collatz sequence as a list of integers.
    """

    if n <= 0:
        raise ValueError("Input must be a positive integer.")

    sequence = []
    while n != 1:
        sequence.append(n)
        if n % 2 == 0:
            n //= 2
        else:
            n = 3 * n + 1
    sequence.append(1)  # Append the final element '1'
    return sequence

def main():
    """
    Main function to handle user input and display the Collatz sequence.
    """
    try:
        n = int(input("Enter a positive integer: "))
        if n <= 0:
            print("Please enter a positive integer.")
            return
        result = collatz_sequence(n)
        print(f"The Collatz sequence for {n} is: {result}")
    except ValueError as e:
        print(f"Invalid input. {e}")
if __name__ == "__main__":
    main()
```

14. (Lucas Number Sequence – Test Case Design)

- Function: Generate Lucas sequence up to n terms.
  (Starts with 2,1, then Fn = Fn-1 + Fn-2)
- Test Cases to Design:
  - Normal: 5 → [2, 1, 3, 4, 7]
  - Edge: 1 → [2]
  - Negative: -5 → Error
  - Large: 10 (last element = 76).
    - Requirement: Validate correctness with pytest.

10.py 11.py 12.py 13.py 14.py

Sem-VI > AI-Assisted > Ass_13 > 14.py > lucas_sequence

```python
def lucas_sequence(n: int) -> list:
    """
    Generate the Lucas sequence up to n terms.

    Args:
        n (int): The number of terms in the Lucas sequence to generate.

    Returns:
        list: A list containing the Lucas sequence up to n terms.
    """
    if n < 1:
        raise ValueError("Input must be a positive integer.")

    sequence = []
    for i in range(n):
        if i == 0:
            sequence.append(2)
        elif i == 1:
            sequence.append(1)
        else:
            next_value = sequence[i-1] + sequence[i-2]
            sequence.append(next_value)
    return sequence

def main():
    """
    Main function to handle user input and display the Lucas sequence.
    """
    try:
        n = int(input("Enter the number of terms for the Lucas sequence: "))
        if n < 1:
            print("Please enter a positive integer.")
            return
        result = lucas_sequence(n)
        print(f"The Lucas sequence up to {n} terms is: {result}")
    except ValueError as e:
        print(f"Invalid input. {e}")
if __name__ == "__main__":
    main()
```

15. (Vowel & Consonant Counter – Test Case Design)

- Function: Count vowels and consonants in string.
- Test Cases to Design:
  - Normal: "hello" → (2,3)
  - Edge: "" → (0,0)
  - Only vowels: "aeiou" → (5,0)
- Large: Long text
- Requirement: Validate correctness with pytest.

10.py 11.py 12.py 13.py 14.py 15.py

Sem-VI > AI-Assisted > Ass_13 > 15.py > ...

```python
def count_vowels_consonants(text: str):
    vowels = "aeiou"
    vowel_count = 0
    consonant_count = 0

    for char in text.lower():
        if char.isalpha():
            if char in vowels:
                vowel_count += 1
            else:
                consonant_count += 1

    return vowel_count, consonant_count

def test_normal():
    assert count_vowels_consonants("hello") == (2, 3)

def test_empty_string():
    assert count_vowels_consonants("") == (0, 0)

def test_only_vowels():
    assert count_vowels_consonants("aeiou") == (5, 0)

def test_only_consonants():
    assert count_vowels_consonants("bcdfg") == (0, 5)

def test_mixed_case():
    assert count_vowels_consonants("HeLLo") == (2, 3)

def test_with_numbers():
    assert count_vowels_consonants("abc123") == (1, 2)

def test_with_special_characters():
    assert count_vowels_consonants("a!b@c") == (1, 2)

def test_large_input():
    text = "hello" * 10000
    assert count_vowels_consonants(text) == (20000, 30000)
```

Running Test using pytest in terminal:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

```
PS C:\Users\rhars\Documents\.Dev\Uni> pytest -v C:\Users\rhars\Documents\.Dev\Uni\Sem-VI\AI-Assisted\Ass_13\15.py
============================================================= test session starts =============================================================
platform win32 -- Python 3.14.0, pytest-9.0.2, pluggy-1.6.0 -- C:\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\rhars\Documents\.Dev\Uni
collected 8 items

Sem-VI/AI-Assisted/Ass_13/15.py::test_normal PASSED                                                                                  [ 12%]
Sem-VI/AI-Assisted/Ass_13/15.py::test_empty_string PASSED                                                                            [ 25%]
Sem-VI/AI-Assisted/Ass_13/15.py::test_only_vowels PASSED                                                                             [ 37%]
Sem-VI/AI-Assisted/Ass_13/15.py::test_only_consonants PASSED                                                                         [ 50%]
Sem-VI/AI-Assisted/Ass_13/15.py::test_mixed_case PASSED                                                                              [ 62%]
Sem-VI/AI-Assisted/Ass_13/15.py::test_with_numbers PASSED                                                                            [ 75%]
Sem-VI/AI-Assisted/Ass_13/15.py::test_with_special_characters PASSED                                                                 [ 87%]
Sem-VI/AI-Assisted/Ass_13/15.py::test_large_input PASSED                                                                             [100%]

============================================================== 8 passed in 0.16s ==============================================================
PS C:\Users\rhars\Documents\.Dev\Uni>
```